Unit–1 Detailed Notes (DSA)

Introduction: Basic Terminologies, Data Structure Operations, Analysis of Algorithm, Asymptotic Notations, Time-Space Trade Off

---

1. Introduction to Data Structure

What is Data?

Data means raw facts and figures.
Examples: 25, Surya, 85 marks, Patna

What is Information?

Processed and meaningful data is called information.
Example: “Surya scored 85 marks in DSA.”

What is Data Structure?

A Data Structure is a method of organizing and storing data in memory so that it can be used efficiently.

Definition:

It is a systematic way to arrange data for easy access, modification, and management.

Examples:

- Array
- Linked List
- Stack
- Queue
- Tree
- Graph

---

2. Need of Data Structure

Data structures are needed because:

1. Efficient storage of data
2. Fast searching and retrieval
3. Easy insertion and deletion

4. Better memory utilization
5. Improves program performance

---

3. Types of Data Structure

A. Primitive Data Structure

Basic built-in data types.

Examples:

- int
- char
- float
- double
- pointer

B. Non-Primitive Data Structure

(i) Linear Data Structure

Elements arranged sequentially.

Examples:

- Array
- Linked List
- Stack
- Queue

(ii) Non-Linear Data Structure

Elements arranged hierarchically or network form.

Examples:

- Tree
- Graph

---

4. Basic Terminologies

1. Data Item

Single unit of value.

Example: Roll number = 101

2. Group Item

Collection of related data items.

Example: Student record = Name + Roll + Marks

3. Elementary Item

Data item that cannot be divided.

Example: Age = 20

4. Entity

Real-world object having attributes.

Example: Student, Employee

5. Attribute

Property of an entity.

Example: Student → Name, Roll No.

6. Field

Smallest unit in record.

Example: Name field

7. Record

Collection of related fields.

Example: Student record

8. File

Collection of records.

Example: All student records file

---

5. Elementary Data Organizations

Data can be organized as:

(a) Character

Single symbol like A, B, 5, #

(b) Field

Group of characters

Example: Name = Surya

(c) Record

Group of related fields

Example:

Roll	Name	Marks
101	Surya	90

(d) File

Collection of records

(e) Database

Collection of related files

---

6. Data Structure Operations

Basic operations performed on data structures:

---

1. Insertion

Adding a new element.

Example: Insert 25 into array.

Time depends on position.

---

2. Deletion

Removing an element.

Example: Delete 10 from list.

---

3. Traversal

Visiting each element one by one.

Example: Print all array elements.

---

4. Searching

Finding location of an element.

Example: Search 50 in array.

---

5. Sorting

Arrange data in ascending/descending order.

Example: 5,2,9 → 2,5,9

---

6. Merging

Combining two lists into one.

---

7. Updating

Modify existing value.

Example: Marks 80 → 85

---

7. Algorithm

Definition:

An algorithm is a finite set of step-by-step instructions used to solve a problem.

Characteristics of Algorithm

1. Input – takes input
2. Output – gives result
3. Definiteness – clear steps
4. Finiteness – must stop
5. Effectiveness – practical steps

Example: Add Two Numbers

1. Start

2. Read A, B
3. Sum = A + B
4. Print Sum
5. Stop

---

8. Analysis of Algorithm

It means measuring efficiency of algorithm.

Done based on:

1. Time Complexity
2. Space Complexity

---

9. Time Complexity

Measures execution time with increasing input size n.

Examples:

- 1 step → Constant time
- n steps → Linear time
- n^2 steps → Quadratic time

---

10. Space Complexity

Memory used by algorithm.

Includes:

- Variables
- Temporary memory
- Recursion stack

---

11. Types of Analysis

Best Case

Minimum time taken.

Worst Case

Maximum time taken.

Average Case

Expected time for random input.

---

12. Asymptotic Notations

Used to express complexity mathematically for large input size.

A. Big O Notation O()

Upper bound / Worst case.

Example:

$$O(n)$$

Means time increases linearly.

Examples:

- Linear Search = O(n)
- Bubble Sort = $O(n^2)$

B. Omega Notation Ω()

Lower bound / Best case.

$$\Omega(1)$$

Example: Search first element in list.

C. Theta Notation Θ()

Tight bound / Exact growth.

$$\Theta(n)$$

Example: Traversing array.

---

13. Common Time Complexities

Complexity	Meaning
O(1)	Constant
O(log n)	Logarithmic
O(n)	Linear
O(n log n)	Efficient sorting
$O(n^2)$	Nested loops
$O(2^n)$	Exponential

---

14. Example of Complexity

Loop Example

```
for(i=1;i<=n;i++)
  printf("%d",i);
```

Runs n times

$$T(n) = n = O(n)$$

Nested Loop Example

```
for(i=1;i<=n;i++)
 for(j=1;j<=n;j++)
  print(i,j);
```

Runs n × n

$$T(n) = n^2 = O(n^2)$$

---

15. Time-Space Trade Off

Means saving time by using extra memory OR saving memory by taking more time.

Definition:

Sometimes faster algorithms need more memory.

Examples:

Example 1: Hash Table

Fast searching O(1), but needs extra memory.

Example 2: Merge Sort

Fast sorting but requires extra array.

Example 3: Linear Search

Less memory but slower than hashing.

---

16. Advantages of Time-Space Tradeoff

1. Improves speed
2. Better performance
3. Useful in large systems

Disadvantages

1. More memory required
2. Costly hardware

Unit–2 Detailed Notes (DSA)

Stacks and Queues

Topics Covered:

- ADT Stack and operations
- Algorithms + Complexity
- Applications of Stack
- Expression Conversion & Evaluation
- ADT Queue
- Types of Queue: Simple, Circular, Priority
- Operations + Algorithms + Complexity

---

1. Stack

Definition

A **Stack** is a linear data structure in which insertion and deletion are performed only at one end called **TOP**.

It follows:

LIFO Principle = Last In First Out

Example: Plate stack

Last plate placed is removed first.

---

2. ADT Stack

ADT = Abstract Data Type

It defines what operations can be done, not implementation.

Basic Operations:

1. Push() → Insert element
2. Pop() → Delete top element
3. Peek()/Top() → View top element
4. isEmpty()

5. isFull()

---

3. Stack Representation

(A) Array Representation

TOP = -1 initially
Stack[0....n-1]

(B) Linked List Representation

Uses nodes + pointers.

---

4. Stack Operations Algorithms

Push Operation

Insert element at top.

Algorithm:

PUSH(STACK, ITEM)

1. If TOP = MAX-1
   Print Overflow
2. Else
   TOP = TOP + 1
   STACK[TOP] = ITEM
3. Stop

Time Complexity:

$$O(1)$$

Pop Operation

Delete top element.

Algorithm:

POP(STACK)

1. If TOP = -1

Print Underflow

2. Else

ITEM = STACK[TOP]

TOP = TOP - 1

3. Return ITEM

Time Complexity:

$$O(1)$$

Peek Operation

If TOP != -1
Return STACK[TOP]

Time:

$$O(1)$$

---

5. Stack Errors

Overflow

When pushing into full stack.

Underflow

When popping from empty stack.

---

6. Applications of Stack

1. Function calls / recursion
2. Undo / Redo
3. Browser backtracking
4. Parentheses checking
5. Expression conversion
6. Expression evaluation
7. DFS in graph

---

7. Expression Conversion

Types of Expressions

Infix:

Operator between operands

Example:

A + B

Prefix:

Operator before operands

Example:

+AB

Postfix:

Operator after operands

Example:

AB+

---

8. Infix to Postfix using Stack

Example:

A + B * C

Priority:

-
    -

Result:

ABC*+

Algorithm

1. Scan infix left to right
2. If operand → output
3. If '(' → push
4. If operator:

pop higher/equal precedence operators
then push current operator
5. If ')' pop till '('
6. Pop remaining operators

Time Complexity:

$$O(n)$$

---

9. Postfix Evaluation

Example:

23*5+

Means:

(2×3)+5 = 11

Algorithm

1. Scan postfix left to right
2. If operand push
3. If operator:
pop two operands
apply operator
push result
4. Final value = answer

Time Complexity:

$$O(n)$$

---

10. Queue

Definition

A **Queue** is a linear data structure in which:

- Insertion at REAR
- Deletion at FRONT

It follows:

FIFO Principle = First In First Out

Example: Ticket line

---

11. ADT Queue

Operations:

1. Enqueue()
2. Dequeue()
3. Front()
4. Rear()
5. isEmpty()
6. isFull()

---

12. Simple Queue

Representation

FRONT = REAR = -1 initially

Enqueue Algorithm

1. If REAR = MAX-1
   Overflow
2. Else if FRONT = -1
   FRONT = 0
3. REAR = REAR + 1
4. Q[REAR] = ITEM

Time:

$$O(1)$$

Dequeue Algorithm

1. If FRONT = -1 or FRONT > REAR
   Underflow
2. ITEM = Q[FRONT]
3. FRONT = FRONT + 1

Time:

$$O(1)$$

Problem in Simple Queue

After deletions, front spaces become wasted.

Solution = Circular Queue

---

13. Circular Queue

Last position connects to first position.

Uses modulo arithmetic.

Next = (REAR + 1) % MAX

Full Condition

(FRONT == (REAR+1)%MAX)

Enqueue in Circular Queue

REAR = (REAR+1)%MAX
Q[REAR]=ITEM

Dequeue

ITEM = Q[FRONT]
FRONT = (FRONT+1)%MAX

Complexity:

$$O(1)$$

Advantages of Circular Queue

1. Better memory utilization
2. No shifting required
3. Fast insertion/deletion

---

14. Priority Queue

Elements inserted with priority.

Higher priority removed first.

If same priority → FIFO.

Example

Element	Priority
A	3
B	1
C	5

Deletion order:

C → A → B

Operations

- Insert by priority
- Delete highest priority

Complexity

Array implementation:

Insertion:

$$O(n)$$

Deletion:

$$O(1)$$

(Depends on implementation)

---

15. Difference Between Stack and Queue

Stack	Queue
LIFO	FIFO
Insert/Delete same end	Different ends
Push/Pop	Enqueue/Dequeue
One pointer TOP	FRONT and REAR

---

16. Complexity Table

Operation	Stack	Queue
Insert	O(1)	O(1)
Delete	O(1)	O(1)
Peek	O(1)	O(1)

Unit–3 Detailed Notes (DSA)

Linked Lists

Topics Covered

- Singly Linked List
- Memory Representation
- Traversing, Searching
- Insertion, Deletion
- Linked Stack & Queue
- Header Nodes
- Doubly Linked List
- Circular Linked List
- Algorithms + Complexity

---

1. Introduction to Linked List

A **Linked List** is a dynamic linear data structure made of nodes.
Each node contains:

1. Data
2. Address (pointer) of next node

Unlike arrays, elements are **not stored in contiguous memory**.

---

2. Why Linked List?

Problems in array:

- Fixed size
- Insertion/deletion costly
- Memory wastage

Solution = Linked List

---

3. Structure of Node

```
struct node
{
  int data;
  struct node *next;
};
```

Each node has:

[data | next]

---

4. Singly Linked List

Each node points to next node only.

START → [10|•] → [20|•] → [30|NULL]

---

5. Representation in Memory

Suppose nodes stored randomly:

Address	Data	Next
100	10	250
250	20	600
600	30	NULL

START = 100

---

6. Basic Operations on Singly Linked List

1. Traversing
2. Searching
3. Insertion
4. Deletion

---

7. Traversing

Visit each node one by one.

Algorithm

```
PTR = START
While PTR != NULL
  Print PTR->data
  PTR = PTR->next
```

Complexity

$$O(n)$$

---

8. Searching

Find item in linked list.

Algorithm

```
PTR = START
While PTR != NULL
  If PTR->data == ITEM
    Found
  PTR = PTR->next
Not Found
```

Complexity

$$O(n)$$

---

9. Insertion in Singly Linked List

(A) Insert at Beginning

```
NEW = new node
NEW->data = ITEM
NEW->next = START
START = NEW
```

Complexity

$$O(1)$$

(B) Insert at End

Move PTR to last node
Last->next = NEW
NEW->next = NULL

Complexity

$$O(n)$$

(C) Insert After Given Node

NEW->next = PTR->next
PTR->next = NEW

Complexity

$$O(1)$$

(after searching node may become O(n))

---

10. Deletion in Singly Linked List

(A) Delete First Node

TEMP = START
START = START->next
Free TEMP

Complexity:

$$O(1)$$

(B) Delete Last Node

Need previous node.

Complexity:

$$O(n)$$

(C) Delete Specific Node

Find previous node, bypass target.

PREV->next = PTR->next
Free PTR

Complexity:

$$O(n)$$

---

11. Advantages of Linked List

1. Dynamic size
2. Easy insertion/deletion
3. No memory wastage
4. No shifting required

---

12. Disadvantages

1. Extra memory for pointer
2. Sequential access only
3. Slower searching than array

---

13. Linked Representation of Stack

Use linked list where insertion/deletion at beginning.

TOP = START

Push

Insert at beginning

Pop

Delete first node

Complexity

$$O(1)$$

---

14. Linked Representation of Queue

Use two pointers:

FRONT and REAR

Enqueue

Insert at end.

Dequeue

Delete from front.

Complexity

$$O(1)$$

(with rear pointer)

---

15. Header Node

A special first node containing metadata, not normal data.

Stores:

- Count of nodes
- Pointer to first node

HEADER → [count=3] → first node

Advantages

1. Simplifies insertion/deletion
2. Stores extra information

---

16. Doubly Linked List

Each node has two pointers:

1. Previous
2. Next

NULL ← [10] ⇄ [20] ⇄ [30] → NULL

Structure:

```
struct node
{
 int data;
 struct node *prev;
```

```
 struct node *next;
};
```

---

17. Operations on Doubly Linked List

Traversal Forward

Move next pointer.

Traversal Backward

Move prev pointer.

Insertion

Update 4 links carefully.

Deletion

Reconnect previous and next nodes.

Complexity

Traversal:

$$O(n)$$

Insertion/Delete at known node:

$$O(1)$$

---

18. Advantages of Doubly Linked List

1. Forward + backward traversal
2. Easier deletion
3. Used in browser history

Disadvantages

1. More memory
2. More pointer updates

---

19. Circular Linked List

Last node points to first node.

START → [10] → [20] → [30]
↑_____|

No NULL at end.

---

20. Operations on Circular Linked List

Traversal

Stop when pointer reaches START again.

Insertion at Beginning

Insert new node before START and update last node link.

Insertion at End

Insert after last node.

Deletion

Delete first / last / specific node carefully.

Complexity

Traversal:

$$O(n)$$

Insert beginning/end (with tail pointer):

$$O(1)$$

---

21. Comparison Table

Type	Pointer(s)	Traversal
Singly	next	Forward
Doubly	prev,next	Both
Circular	next	Circular

Unit–4 Detailed Notes (DSA)

Searching, Sorting and Hashing

Topics Covered

- Linear Search
- Binary Search
- Sorting Objectives
- Selection Sort
- Bubble Sort
- Insertion Sort
- Quick Sort
- Merge Sort
- Heap Sort
- Performance Comparison
- Hashing

---

1. Searching

Searching means finding an element in a list.

Example: Find 25 in array.

[10, 25, 40, 60]

---

2. Types of Searching

1. Linear Search
2. Binary Search

---

3. Linear Search

Check each element one by one.

Example

Find 30 in:

[10,20,30,40]

Compare:

10 ❌
20 ❌
30 ✅

Algorithm

LINEAR(A,n,key)

```
for i=0 to n-1
  if A[i]==key
    return i
return -1
```

Complexity

Best Case:

$$O(1)$$

Worst Case:

$$O(n)$$

Average:

$$O(n)$$

Advantages

1. Simple
2. Works on unsorted data

Disadvantages

Slow for large data.

---

4. Binary Search

Works only on **sorted array**.

Repeatedly divide array into halves.

Example

Find 40 in:

[10,20,30,40,50,60]

Middle = 30

40 > 30 → Search right half

Next middle = 50

40 < 50 → Search left

Found 40

Algorithm

low=0, high=n-1

```
while(low<=high)
 mid=(low+high)/2

 if A[mid]==key return mid
 else if key<A[mid]
     high=mid-1
 else
     low=mid+1
```

Complexity

Best Case:

$$O(1)$$

Worst Case:

$$O(\log\ n)$$

Advantages

1. Very fast

2. Efficient for large data

Limitation

Requires sorted array.

---

5. Difference: Linear vs Binary Search

Linear Search	Binary Search
Works on unsorted data	Needs sorted data
Sequential check	Divide and conquer
O(n)	O(log n)
Simple	Faster

---

6. Sorting

Sorting means arranging data in:

- Ascending order
- Descending order

Example:

5,2,9,1 → 1,2,5,9

Objectives of Sorting

1. Easy searching
2. Better presentation
3. Fast processing
4. Duplicate detection

Properties of Sorting Algorithm

1. Time Complexity
2. Space Complexity

3. Stability
4. In-place or not
5. Simplicity

---

7. Selection Sort

Find minimum element and place at correct position.

Example

[64,25,12,22]

Pass 1: min=12 swap first

[12,25,64,22]

Algorithm

```
for i=0 to n-2
  min=i
  for j=i+1 to n-1
    if A[j]<A[min]
       min=j
  swap(A[i],A[min])
```

Complexity

$$O(n^2)$$

Advantage

Less swaps.

---

8. Bubble Sort

Compare adjacent elements and swap.

Largest bubbles to end.

Example

[5,1,4,2]

After passes:

[1,2,4,5]

Algorithm

```
for i=0 to n-1
 for j=0 to n-i-2
   if A[j]>A[j+1]
     swap
```

Complexity

Worst:

$$O(n^2)$$

Best (optimized):

$$O(n)$$

---

9. Insertion Sort

Insert each element into sorted part.

Example

[5,3,4,1]

Insert one by one:

[1,3,4,5]

Algorithm

```
for i=1 to n-1
 key=A[i]
 j=i-1
```

```
while j>=0 and A[j]>key
  A[j+1]=A[j]
  j--
A[j+1]=key
```

Complexity

Worst:

$$O(n^2)$$

Best:

$$O(n)$$

---

10. Quick Sort

Uses pivot element.

Divide array into smaller and larger parts.

Recursive sorting.

Steps

1. Choose pivot
2. Partition
3. Recursively sort left/right

Complexity

Best/Average:

$$O(n\log\ n)$$

Worst:

$$O(n^2)$$

Advantage

Very fast in practice.

---

11. Merge Sort

Divide array into halves, sort and merge.

Uses divide and conquer.

Complexity

$$O(n\log\ n)$$

(all cases)

Advantage

Stable sort.

Disadvantage

Extra memory required.

---

12. Heap Sort

Uses Binary Heap.

1. Build max heap
2. Delete max repeatedly

Complexity

$$O(n\log\ n)$$

Advantage

No extra array needed.

---

13. Comparison Table

Sort	Best	Worst	Stable	Extra Space
Selection	$O(n^2)$	$O(n^2)$	No	No
Bubble	O(n)	$O(n^2)$	Yes	No
Insertion	O(n)	$O(n^2)$	Yes	No
Quick	O(n log n)	$O(n^2)$	No	Low
Merge	O(n log n)	O(n log n)	Yes	Yes
Heap	O(n log n)	O(n log n)	No	No

---

14. Hashing

Hashing is a technique to store and search data quickly using a **hash function**.

Hash Function

Converts key into index.

Example:

$$h(key) = key \, mod \, 10$$

If key = 35

Index = 5

Example

Store:

25 → 5
32 → 2
18 → 8

---

15. Collision

When two keys map to same index.

Example:

25 and 35 both at index 5

---

16. Collision Resolution

(A) Chaining

Use linked list at same index.

(B) Open Addressing

Find another empty place.

- Linear probing
- Quadratic probing

---

17. Complexity of Hashing

Average Search:

$$O(1)$$

Worst:

$$O(n)$$

Unit–5 Detailed Notes (DSA)

Trees

Topics Covered

- Basic Tree Terminologies
- Binary Tree
- Threaded Binary Tree
- Binary Search Tree (BST)
- AVL Tree
- Tree Operations + Complexity
- Applications of Binary Tree
- B Tree
- B+ Tree

---

1. Introduction to Tree

A **Tree** is a non-linear hierarchical data structure made of nodes connected by edges.

Used to represent hierarchical data like:

- File system
- Organization chart
- Database indexing

---

2. Basic Tree Terminologies

Root Node

Topmost node.

Parent Node

Node having child nodes.

Child Node

Node below parent.

Siblings

Nodes with same parent.

Leaf Node

Node with no children.

Internal Node

Node having at least one child.

Edge

Connection between nodes.

Path

Sequence of nodes connected.

Level

Distance from root + 1

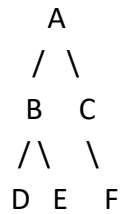
Height of Tree

Maximum level/depth.

Degree of Node

Number of children.

Example



- Root = A
- Parent of D = B
- Leaves = D,E,F
- Height = 3

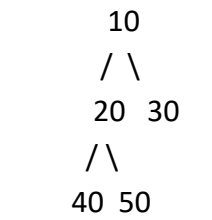
---

3. Binary Tree

A tree where each node has **at most two children**:

- Left child
- Right child

Example



Types of Binary Tree

(1) Full Binary Tree

Every node has 0 or 2 children.

(2) Complete Binary Tree

All levels full except last filled left to right.

(3) Perfect Binary Tree

All internal nodes have 2 children and all leaves same level.

(4) Skewed Tree

All nodes on one side.

---

4. Binary Tree Traversals

(A) Preorder

Root → Left → Right

10 20 40 50 30

(B) Inorder

Left → Root → Right

40 20 50 10 30

(C) Postorder

Left → Right → Root

40 50 20 30 10

Complexity

Traversal visits all nodes:

$$O(n)$$

---

5. Operations on Binary Tree

Insertion

Insert at first available position (level order).

Deletion

Replace with deepest node then delete.

Searching

Traverse all nodes.

Complexity

Search:

$$O(n)$$

---

6. Applications of Binary Tree

1. Expression trees
2. Decision trees
3. Hierarchical data
4. Huffman coding
5. Syntax parsing

---

7. Threaded Binary Tree

Null pointers replaced by links to inorder predecessor/successor.

Purpose: Faster traversal without stack or recursion.

Types

1. Left Threaded
2. Right Threaded
3. Fully Threaded

Advantage

- Better use of NULL links
- Easy inorder traversal

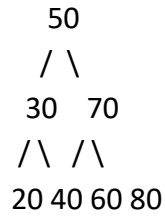
---

8. Binary Search Tree (BST)

A binary tree with rule:

- Left subtree < Root
- Right subtree > Root

Example



---

9. BST Operations

Search

Compare key with root.

- smaller → left

- greater → right

Complexity

Balanced BST:

$$O(\log\ n)$$

Worst skewed:

$$O(n)$$

Insertion

Insert as leaf in proper position.

Deletion Cases

Case 1: Leaf node

Delete directly.

Case 2: One child

Connect child to parent.

Case 3: Two children

Replace with inorder successor/predecessor.

---

10. AVL Tree

Self-balancing BST.

For every node:

$$|\ h_L - h_R\ | \leq 1$$

Where:

- hL = left subtree height
- hR = right subtree height

Why AVL?

Normal BST may become skewed.

AVL keeps height balanced.

---

11. Rotations in AVL Tree

(1) LL Rotation

Right rotation

(2) RR Rotation

Left rotation

(3) LR Rotation

Left then Right

(4) RL Rotation

Right then Left

Complexity

Search / Insert / Delete:

$$O(\log\ n)$$

---

12. B Tree

Multi-level balanced search tree used in databases and disks.

Each node stores multiple keys.

Properties

1. All leaves at same level
2. Node may have many children
3. Balanced tree

Example

[10 | 20 | 30]
/ | | \

Advantages

1. Fewer disk accesses
2. Good for large data

---

13. Operations in B Tree

- Search
- Insert (split node if full)
- Delete (merge/borrow)

Complexity

$$O(\log\ n)$$

---

14. B+ Tree

Advanced form of B Tree.

All records stored in leaf nodes only.

Internal nodes contain keys only.

Leaf nodes linked together.

Advantages

1. Faster range search
2. Sequential access easy
3. Used in DBMS indexing

Difference: B Tree vs B+ Tree

B Tree	B+ Tree
Data in all nodes	Data only in leaf
Search slower	Faster
Leaf not linked	Leaves linked

---

15. Comparison Table

Tree Type	Search
Binary Tree	O(n)
BST	O(log n) avg
AVL	O(log n)
B Tree	O(log n)
B+ Tree	O(log n)

Unit–6 Detailed Notes (DSA)

Graph

Topics Covered

- Basic Graph Terminologies
- Graph Representations
- Graph Search & Traversal
- BFS Algorithm
- DFS Algorithm
- Complexity Analysis

---

1. Introduction to Graph

A **Graph** is a non-linear data structure consisting of:

- **Vertices (Nodes)**
- **Edges (Connections)**

Used to represent networks like:

- Social media
- Road maps
- Computer networks
- Web links

---

2. Graph Notation

A graph is represented as:

$$G = (V, E)$$

Where:

- **V** = set of vertices
- **E** = set of edges

Example:

V = {A,B,C,D}
E = {(A,B),(A,C),(B,D)}

---

3. Basic Graph Terminologies

Vertex

A point/node in graph.

Example: A, B, C

Edge

Connection between two vertices.

Example: (A,B)

Adjacent Vertices

Vertices directly connected.

A and B are adjacent.

Degree of Vertex

Number of edges connected to a vertex.

Example:

If A connected to B,C,D

Degree(A)=3

Path

Sequence of connected vertices.

A → B → D

Cycle

Path starts and ends at same vertex.

A → B → C → A

Connected Graph

Every vertex reachable from another.

Disconnected Graph

Some vertices not connected.

Directed Graph (Digraph)

Edges have direction.

A → B

Undirected Graph

Edges have no direction.

A — B

Weighted Graph

Edges have cost/weight.

A --5-- B

---

4. Example Graph

```
   A
  / \
 B   C
  \ /
   D
```

Vertices = 4
Edges = 4

---

5. Graph Representations

Two main methods:

1. Adjacency Matrix
2. Adjacency List

---

6. Adjacency Matrix

Use 2D array.

If edge exists = 1
Else = 0

For graph:

A-B, A-C, B-D, C-D

	A	**B**	**C**	**D**
A	0	1	1	0
B	1	0	0	1
C	1	0	0	1
D	0	1	1	0

Space Complexity

$$O(V^2)$$

Advantages

1. Easy implementation
2. Fast edge checking

Disadvantages

1. Wastes space for sparse graph

---

7. Adjacency List

Store linked list of neighbors.

Example:

A → B → C
B → A → D
C → A → D
D → B → C

Space Complexity

$$O(V + E)$$

Advantage

Good for sparse graphs.

---

8. Graph Traversal

Traversal means visiting all vertices.

Methods:

1. BFS
2. DFS

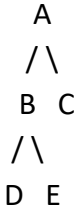
---

9. Breadth First Search (BFS)

Visits vertices **level by level**.

Uses **Queue**.

Example Graph



Start from A

Traversal:

A B C D E

BFS Algorithm

1. Mark start visited
2. Insert into queue
3. While queue not empty
   Remove front vertex
   Visit it
   Insert unvisited adjacent vertices

Complexity

Adjacency List:

$$O(V + E)$$

Adjacency Matrix:

$$O(V^2)$$

Applications of BFS

1. Shortest path in unweighted graph
2. Network broadcasting
3. Social networking levels

---

10. Depth First Search (DFS)

Visits as deep as possible first.

Uses:

- Stack OR
- Recursion

Example

Same graph:

A B D E C

(One possible traversal)

DFS Algorithm (Recursive)

DFS(v)

1. Mark v visited
2. Visit v
3. For each adjacent node u
   If not visited
   DFS(u)

Complexity

Adjacency List:

$$O(V + E)$$

Adjacency Matrix:

$$O(V^2)$$

---

11. Difference Between BFS and DFS

BFS	**DFS**
Queue used	Stack/Recursion
Level wise	Depth wise
Finds shortest path	Not always
More memory sometimes	Less memory sometimes

---

12. Graph Search

Searching means finding a node/path in graph using BFS or DFS.

---

13. Complexity Summary

Operation	Matrix	List
Storage	$O(V^2)$	O(V+E)
BFS	$O(V^2)$	O(V+E)
DFS	$O(V^2)$	O(V+E)

---

14. Applications of Graph

1. GPS Navigation
2. Computer networks
3. Facebook/Instagram connections
4. Web page linking
5. Project scheduling

---

15. Important Exam Questions

2 Marks

1. Define graph.
2. What is degree of vertex?
3. Define path.
4. What is BFS?

---

5 Marks

1. Explain graph terminologies.
2. Explain adjacency matrix and list.
3. BFS vs DFS.

---

10 Marks

1. Explain graph representations with examples.
2. Explain BFS with algorithm and complexity.

3. Explain DFS with algorithm and complexity.
4. Compare BFS and DFS.

---

16. Short Revision Sheet

- Graph = vertices + edges
- Directed graph = arrows
- Undirected = no arrows
- Matrix = 2D array
- List = linked neighbors
- BFS = Queue
- DFS = Stack/Recursion
- BFS/DFS list = O(V+E)

---

17. Memory Trick

Traversal:

BQDS

B = BFS
Q = Queue
D = DFS
S = Stack

---

18. Viva Questions

Q. Which traversal uses queue?
Ans: BFS

Q. Which uses recursion?
Ans: DFS

Q. Which representation saves space?
Ans: Adjacency List

Q. Can graph have cycle?
Ans: Yes

---

19. Final Exam Tip 🔥

Most repeated questions from Unit 6:

1. BFS Algorithm
2. DFS Algorithm
3. Matrix vs List representation
4. Graph terminologies